

Checkers Use Case Descriptions

Use Case 1: Move a piece

Iteration: 1

Primary Actor: Player

Goal in context: The player moves a piece diagonally into an empty square on the board.

Preconditions:

- It is the player's turn
- The selected piece belongs to the player
- The game is in progress
- The destination square must be empty

Trigger: The player selects one of their pieces.

Scenario:

1. The player selects a piece.
2. The player chooses the destination square.
3. The player moves the piece to a destination square.
4. The system updates the new position of the piece on the board.
5. The system checks if the move results in a win.
6. Then it switches to other player's turn.

Postconditions:

- The board is updated with the new position of the piece
- The next player takes their turn

Exceptions: If the move is invalid then system will notify the player and prompts the player to select a valid move.

Priority: High – This is an essential action in the game, without which game cannot progress.

When available: After the game has started.

Frequency of use: Multiple times- once per turn.

Channel to actor: Game interface

Secondary actors: None

Channel to secondary actors: N/A

Open issues: None

Use Case 2: Check for win

Iteration: 2

Primary Actor: System

Goal in context: The system checks if a player has won.

Preconditions:

- A move has been completed
- The board has been updated with the new position of the piece.

Trigger: The system detects that a move has been made.

Scenario:

1. The system checks if one player has no remaining pieces or no valid move left.
2. If this condition is met, the system declares opponent as the winner.
3. The system displays the winner's name.
4. The system ends the game.

Postconditions:

- The winner is announced.
- Game ends.

Exceptions: If the system cannot detect the winner, then the game continues or shows an error message.

Priority: High- For proper game flow, the system must check for a winner.

When available: after every move is completed.

Frequency of use: multiple times- after each move

Channel to actor: System

Secondary actors: None

Channel to secondary actors: N/A

Open issues: Should the game declare a draw if both players cannot move a piece.

Use Case 3: Jump a Piece

Iteration: 1

Primary Actor: Player

Goal in context: The player jumps over an opponent's piece.

Preconditions:

- It is the player's turn
- The selected piece belongs to the player
- The game is in progress
- The destination square for the player must be empty.
- The opponents' piece must be diagonally adjacent to the player's piece.

Trigger: The player selects a piece and attempts to jump over opponent's piece.

Scenario:

1. The player selects a piece.
2. The player chooses the valid destination square for the jump.
3. The player moves the piece to a destination square.
4. The system updates the new position of the piece on the board.
5. The system removes the opponents jumped piece from the board.
6. If another jump is possible, then system allows the player to make another jump.
7. If no further jumps are available, then it switches to other player's turn.

Postconditions:

- The system removes the opponent's piece.
- The board is updated with new position of the piece.
- If another jump is possible, the system allows the player to make another jump
- If no additional jumps are available, the next player takes their turn.

Exceptions: If the move is invalid, the system notifies the player and prompts the player to select a valid move.

Priority: High - It is the important action in the game as it plays key role in determining the winner.

When available: During a players turn

Frequency of use: Multiple times per game, whenever a jump is available.

Channel to actor: Game interface

Secondary actors: None

Channel to secondary actors: N/A

Open issues: None

Use Case 4: Promote to King

Iteration: 1

Primary Actor: Player

Goal in context: The players piece reaches to the last row of opponent side.

Preconditions:

- It is the player's turn
- The selected piece belongs to the player
- The game is in progress
- The destination square must be empty
- The piece is moving to the last row of the opponent's side.

Trigger: The player moves the piece to the last row of opponent's side.

Scenario:

1. The player selects a piece to move.
2. The player moves the piece to the last row of opponent's side.
3. The system checks if the piece has reached the last row.
4. The system promotes the piece to a king, and it marks it as a king.
5. If player has another piece to move, then player continues.
6. If player does not have another piece to move, the turn switches to opponent.

Postconditions:

- The piece is promoted to king.
- The board is updated with the new position of the piece and its status.

Exceptions: If the move is invalid, the system notifies the player and prompts them to select a valid move.

Priority: High: it's important as it impacts gameplay

When available: when the players piece reaches the last row of opponent's side

Frequency of use: multiple times per game, when a piece reaches the last row.

Channel to actor: Game interface

Secondary actors: None

Channel to secondary actors: N/A

Open issues: none

Use Case 5: Surrender

Iteration: 1

Primary Actor: Player

Goal in context: The player surrenders, making the opponent the winner.

Preconditions: The game has started.

Trigger: The player selects “Surrender Game” from the game menu.

Scenario:

1. The player selects “Surrender Game”.
2. The system prompts the player to confirm surrender.
3. If the player confirms, the system declares the opponent as a winner.
4. The system ends the game.

Postconditions:

- The opponent wins
- Game ends.

Exceptions: If the system fails to detect, the game continues or displays an error message.

Priority: Medium- Not required for game progress but enhances the player experience.

When available: Any time after the game has started

Frequency of use: Low: only when player chooses to surrender.

Channel to actor: Game interface

Secondary actors: None

Channel to secondary actors: N/A

Open issues: None